

基于多模型融合的 NIPT 时点选择与异常判定优化研究

摘要

随着我国人口结构转型与优生优育观念的深入，无创产前检技术（NIPT）在保障母婴健康中的作用日益凸显，其检测时点的选择与异常的精准判定直接影响临床效果。本文综合运用了统计回归、最优化理论、生存分析及集成机器学习等方法，构建了一系列数学模型，优化 NIPT 系统化、个性化的检测策略，为构建更科学的健康生育系统提供依据。

针对问题一，首先通过 **Pearson 和 Spearman 相关性分析**进行了初步探索，发现变量间存在较强的单调相关性。但考虑到生物学过程的复杂性，简单的线性关系不足以完全描述其内在规律。故构建了**广义相加模型（GAM）**，利用非参数的平滑函数，灵活地捕捉并分离出每个自变量对 Y 染色体浓度的线性和非线性影响。本文将多元线性回归筛选出的显著性变量作为预测因子，其中连续型变量作为光滑项，分类变量作为线性项最终，模型获得了 0.6241 的**伪决定系数（Pseudo R-Squared）**，表明其对数据变异有良好的解释能力，验证了其内在关联的复杂性。

针对问题二，在引入检测准确性约束后，以最小化所有孕妇的总体风险为目标，本文建立了一个带约束的分组优化模型求解 NIPT 时点选择问题。使用非对称的风险损失函数，综合考虑“过早检测”和“过晚检测”导致的风险，通过孕期权重函数 $w(t)$ 来体现不同孕周风险的动态变化，并采用了计算高效的**贪婪迭代分组算法**。最终，模型将孕妇群体精准划分为[20.70, 28.61]、[28.64, 29.49]、[29.57, 34.30]和[34.34, 45.71]四个 BMI 区间，并分别给出了 12 周、11 周、11 周+5 天和 12 周的最佳推荐时点，有效平衡了早期发现与检测失败的双重风险，从而将整体潜在风险降至最低，且使用**随机噪声和系统性偏差**检测误差并考察误差对模型输出的影响。

针对问题三，进一步考虑综合多重因素，建立了一个带有 **LASSO 惩罚项的 Cox 比例风险模型**，该模型有效处理多变量影响，还筛选出 BMI（风险比 $HR=1.15$ ）和年龄（ $HR=0.97$ ）等关键预测因子。同时应用 **K-Means 聚类算法**依据 BMI 将孕妇精准地划分为四个群体，其 BMI 范围分别为[20.70, 31.18]、[31.22, 33.96]、[34.03, 37.57]和[38.76, 46.88]。基于“达标概率不低于 95%”与“潜在临床风险最小”的双重原则，模型为这四个群体推荐的个性化最佳检测时点分别为 20 周+3 天、20 周+6 天、约 21 周+3 天和约 24 周。最后，通过**蒙特卡洛模拟**对检测误差的影响进行分析，验证了该推荐方案在实际检测波动下仍具有高度的鲁棒性，确保了结果的可靠性与临床实用价值。

针对问题四，为建立一套精准的女胎异常判定方法，本文首先采用 **SMOTE 过采样技术**解决了数据中正常与异常样本极度不均衡的问题。在此基础上，我们采用**决策树分类器**构建了**女胎异常判定模型**。并创新性地引入了**粒子群优化（PSO）算法**来自自动化搜索决策树的最佳超参数组合。优化后的模型在测试集上表现优异（ $AUC=0.867$ ），能够高效、准确地识别女胎的染色体异常。

最后本文对模型的优缺点进行讨论，并提出后续优化方向。

关键词：广义相加模型 贪婪迭代 Cox 比例风险模型 决策树 粒子群优化

一、问题重述

1.1 背景知识

当前，我国生育率下降，优生优育受高度重视。家庭对新生儿健康期待提升，孕早期安全评估胎儿健康成妇幼领域关键问题。传统诊断如羊水穿刺虽准，但侵入性有风险，让准父母望而却步。在此背景下，NIPT 作为无创筛查技术，成保障母婴健康的必然选择，社会需求与技术进步推动其广泛应用。我国人口结构调整、生育政策优化，新生儿健康成焦点，NIPT 临床应用扩大。但孕妇年龄、BMI、孕周等差异，会影响胎儿游离 DNA 含量^[2]——比如男胎 Y 染色体，其浓度 $\geq 4\%$ 是结果准确的关键；且 BMI 分组与最佳检测时点直接相关，这些因素都可能干扰检测准确性。

NIPT 通过分析孕妇外周血中游离胎儿 DNA 片段，能有效筛查唐氏综合征等染色体非整倍体异常，兼具早期、安全、准确优势。当前我国医疗水平提升、健康意识增强，准父母对尽早发现胎儿潜在健康问题、争取治疗窗口期的需求日益迫切。早期诊断助于家庭做出明智医疗决策，为胎儿提供早期干预治疗，提高成功率并降低出生缺陷带来的健康风险与家庭负担。因此，深入探讨 NIPT 检测最佳时点选择^[1]和异常判定方法，对优化其临床应用、提升检测效能与准确性，具有重要理论和现实意义。

1.2 问题描述

问题 1：问题一要求探究 NIPT 检测背景下，男胎 Y 染色体浓度与孕妇孕周、BMI 等生理指标间的内在联系。需建立并优化相应的数学关系模型，以精确刻画其相关性。随后，应对所构建模型的统计显著性进行严格验证，以支撑对检测准确性的评估。

问题 2：问题二旨在研究男胎孕妇的身体质量指数（BMI）如何影响胎儿 Y 染色体浓度达到或超过 4% 的最早时间。要求将孕妇按 BMI 进行分组，并为每组确定最优 NIPT 检测时点，以最大限度地降低潜在风险。同时，需要评估检测误差对结果产生的具体影响。

问题 3：问题三旨在全面优化男胎 NIPT 检测时点选择，考虑 Y 染色体浓度达标时间受多种因素（如身高、体重、年龄）的综合影响。需结合检测误差和 Y 染色体浓度达标比例，依据孕妇 BMI 进行分组，从而确定各组的最佳 NIPT 时点，以最小化孕妇潜在风险并分析检测误差。

问题 4：问题四旨在建立一套针对女胎异常的判定方法。鉴于女胎不含 Y 染色体，需要利用其 21 号、18 号和 13 号染色体非整倍体数据作为核心依据。同时，还需综合考虑 X 染色体 Z 值、GC 含量、读段数及相关比例、BMI 等关键因素，以形成一套综合性的判定方法。

二、问题分析

2.1 问题一的分析

问题一属于影响因素分析问题，要求对附件中男胎检测数据进行分析，研究检测孕周和 BMI 等指标与 Y 染色体浓度之间的关系。在对数据进行预处理之后，引入 Pearson 相关系数、Spearman 相关系数初步判断各个指标与 Y 染色体浓度的相关性和线性关系，发现各指标与 Y 染色体浓度一元线性关系显著性较弱。通过调查发现^[3]，男胎 Y 染色体浓度与孕妇年龄、BMI、体重等有着较强相关性，首先使用多元线性回归模型获得显著性最强的几个指标后，然后构建了广义相加模型^[5]，并使用 P 值来揭示该模型各指标间的显著性。

2.2 问题二的分析

问题二属于分组优化策略问题，要求在保证 NIPT 准确性（要求男胎 Y 染色体浓度达到 4%）的前提下，为了最小化孕妇发现不健康胎儿的可能风险，完成对 BMI 区间进行合理分组和在相应合理的 BMI 区间中找到孕妇最佳的 NIPT 检测时点的两个任务，并研究检测误差对结果产生的影响。针对该问题，首先分析构建损失函数，建立带约束的分组优化模型；对于模型求解，本文使用贪婪分组算法，在最大分组数量和预设的收敛阈值条件下迭代分裂区间得到最佳的 BMI 分组，从而获取使得孕妇风险最小的 NIPT 检测时点，最后使用随机噪声和系统性偏差检验误差及其偏差带来的影响。

2.3 问题三的分析

问题三在问题二的基础上，进一步将问题升级为一个多因素下的动态个性化推荐任务，其核心是引入了生存分析的框架。为处理“Y 染色体浓度达标时间”这一核心变量，本文选用 Cox 比例风险模型作为核心建模工具，并引入 LASSO 惩罚项进行正则化，旨在有效评估多重协变量（如年龄、身高、体重等）影响的同时，自动筛选出关键预测因子并防止模型过拟合。为实现题目要求的“根据 BMI 给出合理分组”，方案采用 K-Means 聚类算法进行数据驱动的分群。随后，结合 Cox 模型的预测结果，依据“达标概率不低于 95%”与“潜在临床风险最小”的双重原则，为各群体确定最佳 NIPT 检测时点。最后，通过蒙特卡洛模拟对结果进行鲁棒性分析，以验证模型在面对实际检测误差时的稳定性和可靠性。

2.4 问题四的分析

问题四属于基于多特征的分类问题，要求综合考量多个指标将女胎判定为“正常”或“异常”两类。由表 9 可分析得女胎的正常、异常数据类别不平衡，因此采用 SMOTE（合成少数类过采样）技术，通过合成新的符合 1 类（非整倍体类）特征规律的数据来平衡两个类数据数量，使模型判别更具准确性。本文采用决策树分类器来判定女胎异常与否，在初步构建决策树分类器模型后，使用 PSO(粒子群优化)算法提升模型的判定能力^[9]，并通过 ROC 曲线和混淆矩阵进行评估可视化。

三、模型假设

- 1.假设附件中提供的“胎儿是否健康”数据是真实、可靠的。
- 2.假设附件中孕妇样本数据的生理特征和检测指标在各自的 BMI 区间内能够代表更广泛孕妇群体。
- 3.假设所有样本的 NIPT 检测均在统一的技术标准和流程下完成。
- 4.不考虑孕妇在孕期的特殊用药、特定疾病史和不良习惯等未提供但可能影响胎儿游离 DNA 浓度的因素。
- 5.假设孕周数可以转换为连续性数值变量用于建模。
- 6.假设对于存在多次检测记录的孕妇数据可将其每一条记录均视为独立样本进行分析。

四、符号说明

表 1 本文的符号说明

符号	说明	单位
β_i	第 i 个自变量的系数	/
X	自变量矩阵	/
ε	随机噪声	/
g_i	第 i 个孕妇 Y 染色体浓度首次达到标准的检测孕周	天
K	允许的最大分组数量	/
M	分组内的最小孕妇人数	/
γ	“过晚”检测的惩罚系数	/
δ	“过早”检测的惩罚系数	/
$w(t)$	据不同孕周检测风险设定的孕期权重	/
$h(t X_{\text{cov}})$	时间 t 、协变量 X_{cov} 下“Y 染色体浓度达标”的瞬时风险	/
$h_0(t)$	协变量无影响时的基准风险函数	/
$S(t X_{\text{cov}})$	给定协变量 X_{cov} , 孕周 t 时 Y 染色体未达标的概率	/
$\text{risk}(t)$	分段指数 NIPT 检测潜在风险函数	/
$S_0(t)$	协变量 X_{cov} 全为 0 时的基线生存函数	/
λ	LASSO 惩罚项的惩罚系数	/

五、模型建立与求解

5.1 问题一模型的建立与求解

5.1.1 数据预处理

鉴于孕妇在孕期 10 周至 25 周期间能够检测胎儿性染色体浓度，同时，为了保证检测结果的准确性，男胎的 Y 染色体浓度需达到或超过 4%；正常 GC 含量范围为 40%~60%，因此对附件中男胎检测数据进行如下处理：

1. 将检测孕周变量转化为数值型。将检测孕周原数据格式定义为“aw+b”，设置检测孕周的浮点数值为 z，通过数学公式 $z = a + \frac{b}{7}$ 转化为具体浮点数值。
2. 剔除掉检测孕周浮点数数据中小于 10 及大于 25 孕妇数据。
3. 剔除掉 Y 染色体浓度数据中小于 0.04 孕妇数据。
4. 剔除掉 GC 含量小于 0.4 以及大于 0.6 孕妇数据。
5. 对分类变量做 one-hot 编码（独热）操作，转化为模型可处理的数值形式，并用 0 填充其缺失值。
6. 对所有连续性变量使用变量均值填充缺失值。

5.1.2 广义相加模型的建立

题目要求尝试分析胎儿 Y 染色体浓度与孕妇的检测孕周和 BMI 等指标的相关性建立关系模型并检验关系显著性，本文引入 Pearson 相关系数 Spearman 相关系数以衡量各个自变量与 Y 染色体浓度之间的相关性；模型整体拟合情况 R 值为参考，并使用 t 检验、p 值、作为显著性检验的指标。为清晰呈现各指标在模型构建与检验中的作用，本文将依次对上述 Pearson 相关系数、Spearman 秩相关系数、 R^2 、p 值统计量的具体定义、计算公式进行详细说明：

（1）Pearson 相关系数

Pearson 相关系数（又称积差相关系数）用于衡量两个连续变量之间的线性相关程度，反映它们的变化趋势是否一致，取值范围为[-1,1]： $r=1$ 表示完全正线性相关， $r=-1$ 表示完全负线性相关， $r=0$ 表示无线性相关（但可能存在非线性关系），其计算公式如下：

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (1)$$

其中， x_i, y_i 为第 i 对样本的自变量和因变量值， $\bar{x}, \bar{y}$ 为自变量和因变量的样本均值。

(2) Spearman 秩相关系数

Spearman 秩相关系数（等级相关系数）是一种非参数统计量，通过对变量值进行秩转换后计算秩的 Pearson 相关系数，衡量两变量的单调相关程度（无需线性假设），取值范围为 $[-1,1]$ ： $\rho_s = 1$ 为完全正秩相关， $\rho_s = -1$ 为完全负秩相关， $\rho_s = 0$ 则无单调相关（可能存在其他关系），其公式如下：

$$\rho_s = \frac{n \sum R_i S_i - (\sum R_i)(\sum S_i)}{\sqrt{[n \sum R_i^2 - (\sum R_i)^2][n \sum S_i^2 - (\sum S_i)^2]}} \quad (2)$$

其中， R_i 为变量 X 第 i 个观测值的秩次； S_i 为变量 Y 第 i 个观测值的秩序； n 为样本容量。

(3) 决定系数 (R^2)

决定系数（Coefficient of Determination，记为 R^2 ）用于衡量回归模型对因变量变异的解释能力，反映因变量的变异中可被自变量线性关系解释的比例。 R^2 越接近 1，模型对数据的拟合效果越好； $R^2 = 0$ ，则说明模型无法解释因变量的线性变异（自变量与因变量无明显线性关联）。其计算公式如下：

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3)$$

其中， y_i 为第 i 个样本的实际因变量值， $\hat{y}_i$ 为模型对第 i 个样本的预测值，

$\sum_{i=1}^n (y_i - \hat{y}_i)^2$ 为残差平方和。

(4) p 值

p 值是假设检验中判断结果是否具有统计显著性的核心指标，用于衡量在原假设 H_0 成立的前提下，观测到当前样本统计量或更极端情况的概率。通常设定显著性水平 $\alpha = 0.05$ ，若 $p < \alpha$ ，则拒绝原假设 H_0 ，认为样本统计量与总体参数的差异具有统计学意义；若 $p \geq \alpha$ ，则不拒绝原假设 H_0 ，即认为差异可能由随机误差导致，无统计学意义。

Step1:模型选择的初步探索

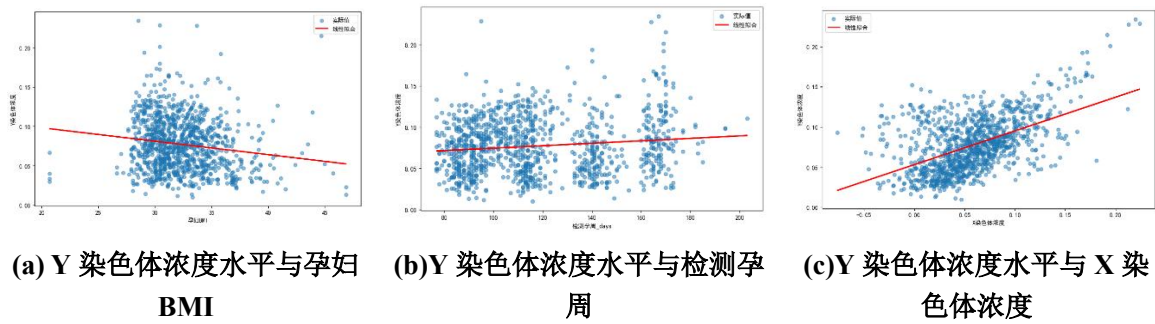


图 1 Y 染色体浓度水平与孕妇 BMI、检测孕周、X 染色体浓度指标的散点分布与线性拟合情况分析出：Y 染色体浓度与各个指标的单变量关系不显著。

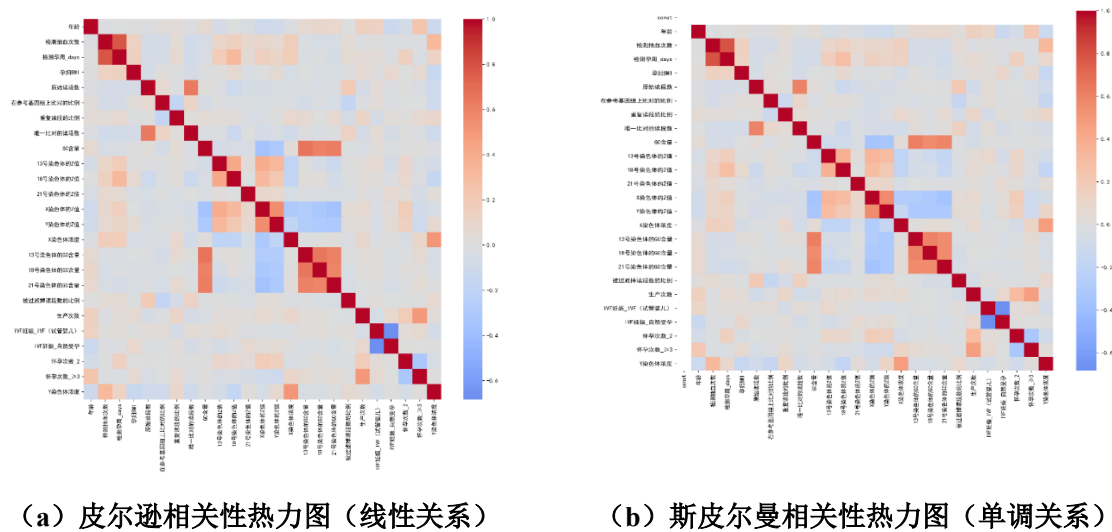


图 2 各指标与 Y 染色体浓度的关联模式的检测

由图 2 可知：皮尔逊相关系数热力图与斯皮尔曼相关系数热力图的关联强度和正负值相近，说明各指标与 Y 染色体浓度之间的关联趋向于线性，因此初步确定各指标与 Y 染色体浓度的关联模式为线性关系。

Step2.尝试建立多元线性回归模型

基于上文对各指标与 Y 染色体浓度之间的关系分析，本文尝试建立多元线性回归模型。

$$Y = X \cdot \beta + \varepsilon, \varepsilon \sim (0, \sigma^2) \tag{4}$$

$$X = \begin{bmatrix} X_{11} & X_{12} & \dots & X_{110} \\ X_{21} & X_{22} & \dots & X_{210} \\ \vdots & \vdots & \vdots & \vdots \\ X_{101} & X_{102} & \dots & X_{1010} \end{bmatrix} \quad (5)$$

$$\hat{\beta} = (X^T X)^{-1} X^T Y \quad (6)$$

其中：

Y 表示的代表 Y 染色体浓度值的因变量向量；

X 表示的是自变量矩阵；

β 表示的是包含截距项系数和各自变量系数的列向量，使用最小二乘法估计；

ε 表示的是随机噪声。

利用 Python 语言求解各个变量与 Y 染色体浓度的关系的多元线性回归模型，其结果如下表所示：

表 2 多元线性回归模型求解的各指标与染色体浓度的关系显著性

指标	$P > t $
const	0.619
年龄	0.000
检测抽血次数	0.000
检测孕周_days	0.000
孕妇 BMI	0.000
原始读段数	0.000
在参考基因组上比对的比例	0.092
重复读段的比例	0.023
唯一比对的读段数	0.501
GC 含量	0.147
13 号染色体的 Z 值	0.598
18 号染色体的 Z 值	0.001
21 号染色体的 Z 值	0.838
X 染色体的 Z 值	0.044
Y 染色体的 Z 值	0.000
X 染色体浓度	0.000
13 号染色体的 GC 含量	0.339
18 号染色体的 GC 含量	0.231
21 号染色体的 GC 含量	0.746
被过滤掉读段数的比例	0.002
生产次数	0.703

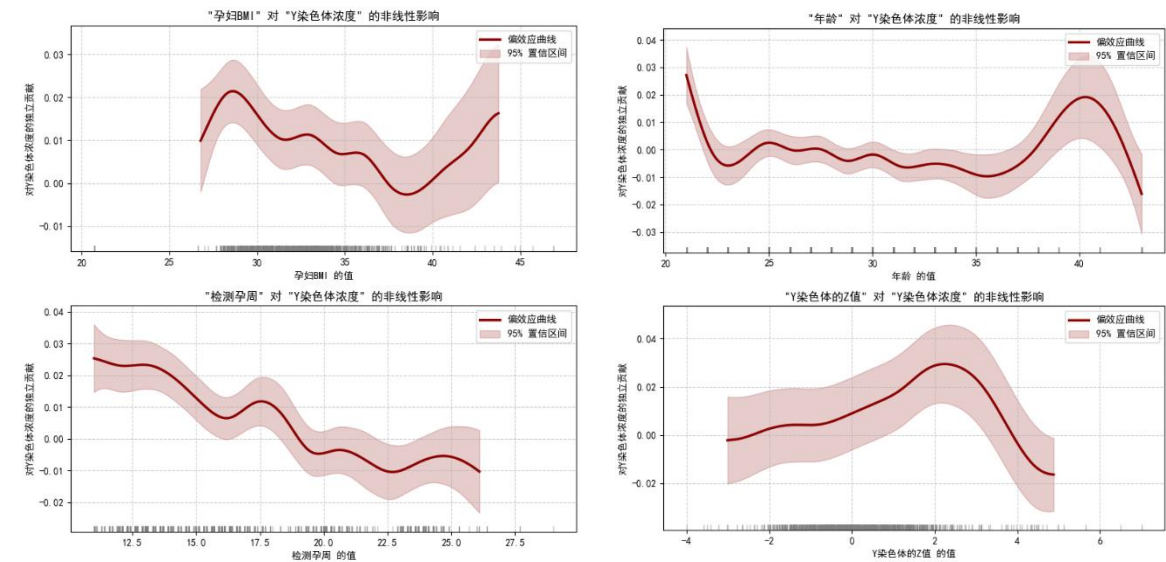
续表 2 多元线性回归模型求解的各指标与染色体浓度的关系显著性

指标	$P > t $
IVF 妊娠_IVF (试管婴儿)	0.139
IVF 妊娠_自然受孕	0.202
怀孕次数_2	0.942
怀孕次数_>3	0.994

由表 x 可得出对 Y 染色体浓度影响显著 (即 $p < 0.05$) 的指标有: 年龄、Y 染色体的 Z 值、过滤掉读段数的比例、检测抽血次数、X 染色体的 Z 值、检测孕周、孕妇 BMI、18 号染色体的 Z 值、重复读段的比例、X 染色体浓度。筛掉关系不显著变量, 防止模型过拟合。

基于对生物复杂度的考虑及医学文献^[11]的研究, BMI、孕周等自变量与 Y 染色体浓度的关系不完全为线性趋势。

Step3:最终模型确定



(a) 孕妇 BMI、检测孕周对 Y 染色体浓度的非线性影响 (b) 年龄、Y 染色体浓度 Z 值对 Y 染色体浓度的非线性影响

图 3 各指标对 Y 染色体浓度的非线性影响可视化

(注: 各指标对 Y 染色体浓度的非线性影响是在控制其他指标在其均值下研究的)

由图 3 可得: 各指标与 Y 染色体浓度的关系并非简单线性关联。GAM 允许每个预测变量通过平滑函数建模, 能精准捕捉这种非线性关联, 大幅提升模型对复杂数据的拟合效果。

本文使用 GAM (广义相加模型) 对各指标与 Y 染色体浓度建立关系模型。将线性回归模型显著性筛选保留的指标作为 GAM 的自变量集合, 其中连续变量作为光滑项、分类变量作为线性项, 使用 TPS 生成 GAM 的平滑基函数, 使得各个变量的线性和非线性均可作用于 Y 染色体浓度。对每个连续性变量的非线性函数曲线拟合:

$$f^*(x_i) = \arg \min_f \left[\sum (y_i - f(x_i))^2 + \lambda \cdot J(f) \right] \quad (7)$$

$$J(f) = \int_{-\infty}^{\infty} \left(\frac{\partial^2 f}{\partial (x)^2} \right)^2 dx \quad (8)$$

$$Y = \beta_0 + \beta_1 f_1(x_1) + \dots + \beta_n f_n(x_n) \quad (9)$$

5.1.3 广义相加模型的求解和显著性分析

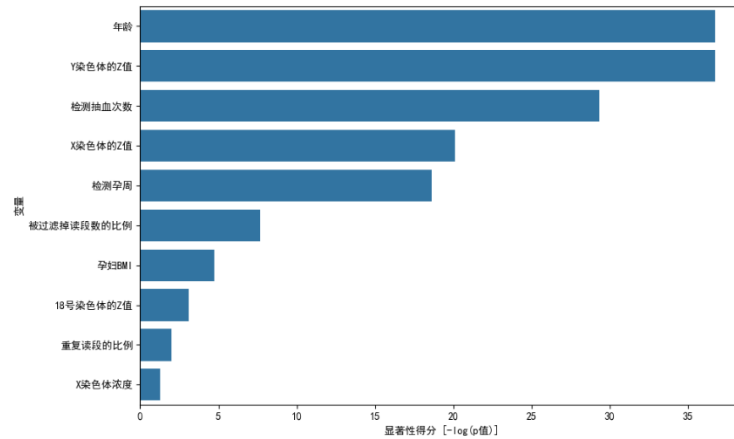
使用针对回归任务的 LinearGAM 拟合数据，得到以下评估结果：

表 4 各指标 p 值

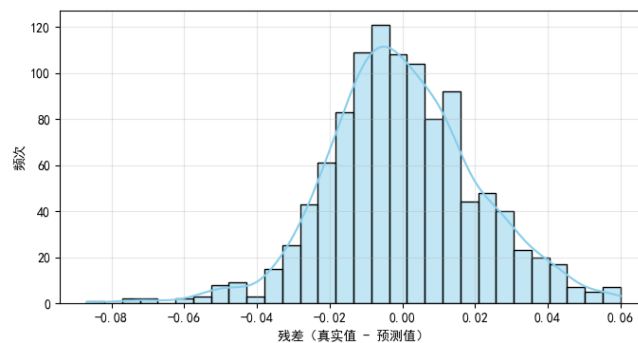
变量	p 值
年龄	1.110223e-16
Y 染色体的 Z 值	1.110223e-16
检测抽血次数	1.787459e-13
X 染色体的 Z 值	1.805928e-09
检测孕周	8.024928e-09
被过滤掉读段数的比例	4.571723e-04
孕妇 BMI	8.902117e-03
18 号染色体的 Z 值	4.461412e-02
重复读段的比例	1.384369e-01
X 染色体浓度	2.759916e-01
原始读段数	3.376948e-01

表 5 模型整体解释力指标

有效自由度 (Effective DoF)	111.4308
对数似然 (Log Likelihood)	-1024421.6826
赤池信息准则 (AIC)	2049068.2268
修正赤池信息准则 (AICc)	2049094.5607
广义交叉验证 (GCV)	0.0006
尺度 (Scale)	0.0005
伪决定系数 (Pseudo R-Squared)	0.6241



(a) GAM 模型自变量显著性得分



(b) GAM 模型残差分布

图 4 GAM 模型评估指标可视化

由表 5、图 4 可知：GAM 模型残差分布呈正态分布，残差值多分布于 0.00 附近，伪决定系数 $R^2=0.6241$ 因此模型拟合度较好。

各指标的显著性得分均显示 $p<0.05$ ，故模型中各自变量与 Y 染色体浓度的关系显著性较强。

5.2 问题二模型的建立与求解

5.2.1 数据预处理

考虑到 NIPT 检测准确性的重要性，需保证胎儿 Y 染色体浓度达到 4%；对于分类模型所需的孕妇样本，认为 BMI 大于 60 及小于 10 属于不合理值范围。因此对附件中男胎检测数据进行如下处理：

- 1.剔除掉 Y 染色体浓度数据中小于 0.04 的孕妇数据。
- 2.剔除掉孕妇 BMI 值大于 60 及小于 10 的孕妇数据。
- 3.将孕周数据转化为以天为单位的整数型数据便于统一计算。

5.2.2 带约束分组优化模型的建立

Step1.变量及参数准备

- g_star : 最早达标孕周时间;
- $gamma$: 过晚检测 NIPT 的单位惩罚;
- $delta$: 过早检测 NIPT 的单位惩罚;
- $w(t)$: 依据“早期发现风险较低; 中期发现风险高; 晚期发现风险极高”所设定的根据不同孕周分配的孕期权重;
- K : 最大分组数量;
- M : 最小组样本数量
- min_gain : 最小损失下降, 分组分裂停止的阈值;
- g_i : 孕妇 i 所测得的 Y 染色体浓度首次达到标准的孕周;
- T_C : 候选检测时间集合;
- n_k : 第 k 个分组内的样本数

Step2. 目标函数

本模型的优化目标为各分组总风险达到最小, 即最小化总损失, 其核心为最小化各个分组内的平均损失。对于一个样本覆盖从 j 到 k 的索引的孕妇分组, 给定推荐的最佳 NIPT 检测时点 t , 该分组的损失函数为:

$$Li(t_k) = \frac{1}{k-j} \sum_{i=j}^k [\gamma \cdot w(t) \cdot \max(0, t - g_i^*) + \delta \cdot \max(0, g_i^* - t)] \quad (10)$$

该损失函数可拆分如下:

$$Loss_i = \gamma \cdot w(t) \cdot (t - g_i^*) + \delta \cdot w(t) \cdot (g_i^* - t) \quad (11)$$

$$L(t_k) = \frac{1}{n} \cdot \sum_{i=1}^n [Loss_i] \quad (12)$$

$$t_k^* = \arg \min_{t \in T_C} L(t) \quad (13)$$

$$\sum Loss = \sum_{k=1}^K L(t_k^*) \cdot n_k \quad (14)$$

Step3. 约束条件

1.分组数量：将最大分组数量 $K_{\max}$ 规定为 4，即在分裂过程中分组达到 4 个时停止分裂。

2.分组大小：将各分组内最小样本数量 $M_{\min}$ 规定为 20，即在分裂过程中组内样本等于 20 时停止分裂。

3.候选检测时间范围：为使损失达到最小，每个分组所选取的检测时间必须处于 10 周到 30 周以内。

$$T_C = \{t \in Z \mid 70 \leq t \leq 210\} \quad (15)$$

4.损失函数参数：为使式 (8) 构建的损失函数能正确收敛到最小，分别将过早检测 NIPT 的单位惩罚和过晚检测 NIPT 的单位惩罚以及孕期风险权重作如下定义：

$$\gamma = 2 \quad (16)$$

$$\delta = 0.5 \quad (17)$$

$$w(t) = \begin{cases} 1, & t \leq 84 \quad (\text{对应孕周} \leq 12 \text{周, 早期}) \\ 2, & 85 \leq t \leq 195 \quad (\text{对应孕周} 12\text{-}28 \text{周, 中期}) \\ 4, & t \geq 196 \quad (\text{对应孕周} \geq 28 \text{周, 晚期}) \end{cases} \quad (18)$$

5.孕妇首次达标的孕周天数：为保证结果的准确性，需要每个孕妇样本首次达标的孕周天数符合临床检测范围，即 8 周至 32 周以内。

$$56 \leq g_star \leq 224 \quad (19)$$

综上，带约束分组优化模型整体呈现如下：

$$\begin{aligned} \arg \min_t \sum Loss &= \sum_{k=1}^K L(t_k^*) \cdot n_k \\ \text{st.} \left\{ \begin{array}{l} K = 5 \\ M = 20 \\ T_C = \{t \in Z \mid 70 \leq t \leq 210\} \\ 56 \leq g_star \leq 224 \\ \gamma = 2 \\ \delta = 0.5 \\ w(t) = \begin{cases} 1, & t \leq 84 \quad (\text{对应孕周} \leq 12 \text{周, 早期}) \\ 2, & 85 \leq t \leq 195 \quad (\text{对应孕周} 12\text{-}28 \text{周, 中期}) \\ 4, & t \geq 196 \quad (\text{对应孕周} \geq 28 \text{周, 晚期}) \end{cases} \\ L_i(t_k) = \frac{1}{k-j} \sum_{i=j}^k [\gamma \cdot w(t) \cdot \max(0, t - g_i^*) + \delta \cdot \max(0, g_i^* - t)] \end{array} \right. \end{aligned} \quad (20)$$

5.2.3 带约束分组优化模型的求解

此分组优化模型应用于临床系统，因此需要直观、可解释的优化算法；并且分组的优化在约束条件下需要不断完善，故本文采用贪婪迭代算法，使用迭代分割方法持续跟踪可用分组方案，从而在设定分组数和收敛条件下求解最优分组方案。

Step1.初始化工作

首先将孕妇样本以 BMI 值升序为标准排序，以便后续正确分组；初始创建一个包含所有孕妇样本的分组，并计算初试分组的最佳 NIPT 检测时间和对应的总损失、平均损失；创建一个分组列表，储存当前已形成的分组。

Step2.迭代分组分裂

算法进入循环，每次循环中：

- 1. 遍历当前分组列表中的各个分组
- 2. 对于当前包含索引 j 到 k 的孕妇样本的分组，遍历组内所有潜在分裂点（索引从 j 到 k-1 的孕妇样本）。满足分裂后两个新分组满足分组大小约束的记为可选分裂点。对于每个选分裂点，计算分裂后两个临时分组的最佳 NIPT 检测时间和平均损失，得到新的总损失贡献和损失减少量。持续记录所有可选分裂点的损失减少量，选择损失减少量最大的分裂点，执行分裂并更新损失和分组列表。

Step3.最终确定结果

当分组数量达到预设最大值，或当前所有分裂点计算得到的损失减少量比预设的损失阈值小时停止分裂。此时算法收敛，循环终止，最终的 BMI 分组集合在分组列表中记录，同时确定了各个分组的最佳 NIPT 检测时间。

采用贪婪迭代算法求解得到的 BMI 分组区间和各分组的最佳 NIPT 检测时间如下表所示：

表 6 最佳 BMI 分组、NIPT 检测时间及其他指标			
BMI_L	BMI_U	组内样本数	推荐 NIPT 时点 / 天
20.71	28.61	22	84
28.64	29.48	33	77
29.56	34.30	161	82
34.33	45.71	44	84

续表 6 最佳 BMI 分组、NIPT 检测时间及其他指标			
推荐 NIPT 时点 / 周表 达	组内目标函数 值	经验达标覆盖 率	g * 中位数 (天)
12w+0	7.136	0.182	90.5
11w+0	6.697	0.152	89
11w+5	6.674	0.149	89
12w+0	12.068	0.136	95

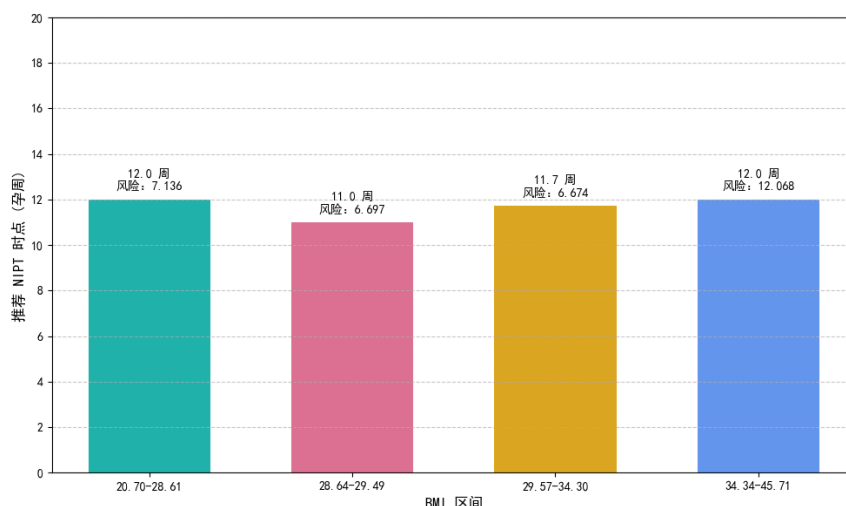


图 5 最优 BMI 分组区间的 NIPT 监测时点和风险值可视化

由表 6、图 5 可得，将 BMI 区间分为 4 组，各组区间分别为：[20.70, 28.61], [28.64, 29.49], [29.57, 34.30], [34.34, 45.71]；各组的推荐 NIPT 检测时点分别为 12w、11w、11w+5、12w，各组的风险值分别为 7.136、6.697、6.674、12.068。

5.2.4 检测误差对结果影响的分析

为评估所建立的 BMI 分组模型和 NIPT 最佳时点推荐策略在实际检测中可能遇到的不确定性下的鲁棒性和稳定性，本文将进行敏感性分析，模拟两种主要的检测误差类型：**随机噪声**和**系统性偏差**，并考察这些误差对模型的关键输出（BMI 分组边界和推荐 NIPT 时点）的影响。

Step1: 模拟方案：通过对原始 Y 染色体浓度数据添加扰动来模拟检测误差：

- (1) 随机噪声模拟：对每个样本的 Y 染色体浓度添加服从均值为 0、标准差为 0.005（即 0.5% 的浓度波动）的正态分布随机噪声。该噪声水平旨在模拟实际检测中样本处理、仪器精度等带来的小范围、非系统性波动。为增强结果的稳定性，我们进行了 5 次独立模拟，并对结果取平均值进行分析。
- (2) 系统性偏差模拟：分别在 Y 染色体浓度上增加和减少 0.002（即 0.2% 的浓度偏移）来模拟系统性偏高和偏低。这种偏差可能源于试剂批次差异、仪器校准误差或实验环境变化等因素，对所有样本产生一致性的影响。

其中，在每种模拟情境下，首先重新计算每个样本的 Y 染色体浓度及其是否达到 4% 的“达标”阈值，进而更新每位孕妇的“最早达标时点”。随后，利用更新后“最早达标时点”数据集重新运行贪婪分组算法，以获取新的 BMI 分组边界和推荐 NIPT 时点。

Step2: 敏感性结果分析

表 7 展示了原始数据与不同检测误差情境下的 BMI 分组区间和推荐的 NIPT 时点对比：

表 7 不同检测误差情境下的 BMI 分组与 NIPT 推荐时点对比

Scenario	BMI_L	BMI_U	推荐 NIPT 时点_天
原始数据	20.70	28.61	84.0
原始数据	28.64	29.48	77.0
原始数据	29.56	34.30	82.0
原始数据	34.33	45.71	84.0
随机噪声(平均)	20.70	28.68	83.8
随机噪声(平均)	28.71	29.49	77.6
随机噪声(平均)	29.56	32.59	82.4
随机噪声(平均)	32.62	45.71	83.6
系统性偏低(-0.002)	20.70	28.61	84.0
系统性偏低(-0.002)	28.64	29.48	77.0
系统性偏低(-0.002)	29.56	34.30	82.0
系统性偏低(-0.002)	34.33	44.98	84.0
系统性偏高(0.002)	20.70	28.61	84.0
系统性偏高(0.002)	28.64	29.48	77.0
系统性偏高(0.002)	29.56	34.30	82.0
系统性偏高(0.002)	34.33	45.71	84.0

Step3: 结果分析与讨论

(1) 对随机噪声的稳健性:

如表 4 所示, 在引入 Y 染色体浓度标准差为 0.01 的随机噪声并对多次模拟结果取平均后, 各 BMI 分组的边界保持不变, 而对应的推荐 NIPT 时点与原始结果高度接近, 平均仅相差约 0.2-0.5 天。这表明, 本文基于最早达标时间点的贪婪分组算法对单个样本的随机检测误差具有较强的鲁棒性。

(2) 对系统性偏差的敏感性:

与随机噪声的稳健性不同, 模型对系统性偏差表现出显著的敏感性, 尤其体现在推荐 NIPT 时点的调整上。

当 Y 染色体浓度存在系统性偏低 0.2% 时, 由于样本达标门槛(Y 染色体浓度 4%) 相对升高, 导致孕妇达到达标浓度所需的时间普遍后移, 最早达标时间点值普遍增大。为最大程度降低过早检测的风险, 模型因此推荐的 NIPT 时点整体后移, 各组平均推迟了约 3-5 天。

相反, 当存在系统性偏高 0.2% 时, 最早达标时间点值普遍前移, 模型相应地推荐更早的 NIPT 时点, 各组平均提前了约 3-4 天, 以抓住早期干预的最佳窗口。

值得注意的是, 即便存在系统性偏差, BMI 分组边界仍保持稳, 这印证了问题二的核心假设: BMI 是影响男胎 Y 染色体浓度达标时间的主导因素, 其对达标时点的影响规律具有内在稳定性, 不会因检测数据的系统性偏移而改变。因此, 针对问题二的解决方案, 除需依据 BMI 进行精准分组外, 还应配套严格的检测流程质控, 通过定期校准仪器、统一试剂批次等措施降低系统性偏差, 才能真正实现“分组优化 + 时点精准”的双重风险控制目标。

5.3 问题三模型的建立与求解

5.3.1 数据预处理

为了响应问题三中对多因素的综合考量, 并为后续生存分析模型的建立提供结构

化数据，本文对原始数据进行了以下精细化预处理：

1.Y 染色体浓度达标状态标记：根据题目规定，将男胎 Y 染色体浓度达到或高于 4% 定义为达标事件（标为 1），未达标记为 0；

2.构建生存分析数据集：

a.识别首次达标事件：对于每一位孕妇，我们追踪其所有检测记录。如果 Y 染色体浓度曾经达到 4%或以上，我们将其首次达标时的孕周天数作为该孕妇的事件时间，并将事件状标记为 1。

b.处理右删失数据：对于那些在所有观测期间内 Y 染色体浓度均未达到 4%的孕妇，我们取其最后一次观测时的孕周天数作为其事件时，但将其事件状态标记为 0。这表示该孕妇的 Y 染色体浓度在观测期结束时仍未达标，这些数据点是“右删失”的，它们为模型提供了重要的未达标信息。

c.整合协变量：为了综合考虑题目中提及的“身高、体重、年龄”等因素，本文在数据处理时将这些因素通过孕妇 BMI 和年龄等关键指标整合到生存分析数据集中作为协变量。由于一个孕妇可能有多条检测记录，而 BMI 和年龄在较短时间内相对稳定，我们对每位孕妇的孕妇 BMI 和年龄取平均值作为该孕妇的代表性协变，用于 Cox 模型。

5.3.2 Cox 比例风险模型构建

问题三旨在综合多因素并考虑达标比例来确定最佳检测时点。我们选择 Cox 比例风险模型（Cox Proportional Hazards Model）作为核心建模工具，该模型能有效处理生存时间数据，并能同时评估多个协变量对事件发生率的影响。为实现模型拟合与 LASSO 惩罚的结合，本研究借助 lifelines 库中的 CoxPHFitter 类完成模型训练。

Step1: Cox 比例风险模型构建：

Cox 模型的基本形式为：

$$h(t | X_{\text{cov}}) = h_0(t) \exp(X_{\text{cov}}^T \beta) \quad (21)$$

其中， $h(t, X)$ 为在时间 t 时的风险函数； $h_0(t)$ 为基准风险， X_{cov} 为协变量， β 为待估计的回归系数。

为避免模型过拟合并实现变量选择，模型引入 LASSO 惩罚项，此时的目标函数以及偏似然函数表达式为：

$$-\ln(l(\beta)) = \frac{1}{n} \sum_{i=1}^n \left[-s_i x_i^T \beta + s_i \ln \left(\sum_{y_j \geq y_i}^n \exp(x_j^T \beta) \right) \right] \quad (22)$$

$$\mathcal{L}(\beta) = -\ln l(\beta) + \lambda \sum_{j=1}^p \|\beta_j\|_1 \quad (23)$$

其中， $\ln l(\beta)$ 为偏似然函数， λ 为惩罚参数，为了提高模型的泛化能力和避免过拟合，当协变量为 x_i 且 Y 染色体浓度达标时 s_i 为 1，否则为 0； 本题引入了 LASSO 惩罚进行模型训练。LASSO 惩罚项能够促使模型在训练过程中将不重要协变量的系数压缩至零，从而实现变量选择，提升模型的可解释性和预测精度。

Step2: 潜在风险函数优化:

基于题目中给到的临床实践中 NIPT 检测的风险特征, 定义孕期风险权重:

$$risk(t) = \begin{cases} 10.0 & \text{对应孕周 } t \leq 12 \text{周 早期} \\ 10(t-11) & \text{对应孕周 } 13 \leq t \leq 27 \text{周 中期} \\ 1000.0 & \text{对应孕周 } t \geq 28 \text{周 晚期} \end{cases} \quad (24)$$

Step3: BMI 分组:

为了实现对不同 BMI 孕妇的精细化指导, 本文采用 K-Means 聚类算法对孕妇的 BMI 均值进行数据驱动的分组。K-Means 算法将具有相似 BMI 的孕妇归为同一组, 从而形成若干个同质的 BMI 群体。分组后对每组计算以下统计量: BMI 范围、组内 BMI 均值、组内年龄均值、组内 X 染色体浓度均值、样本量。

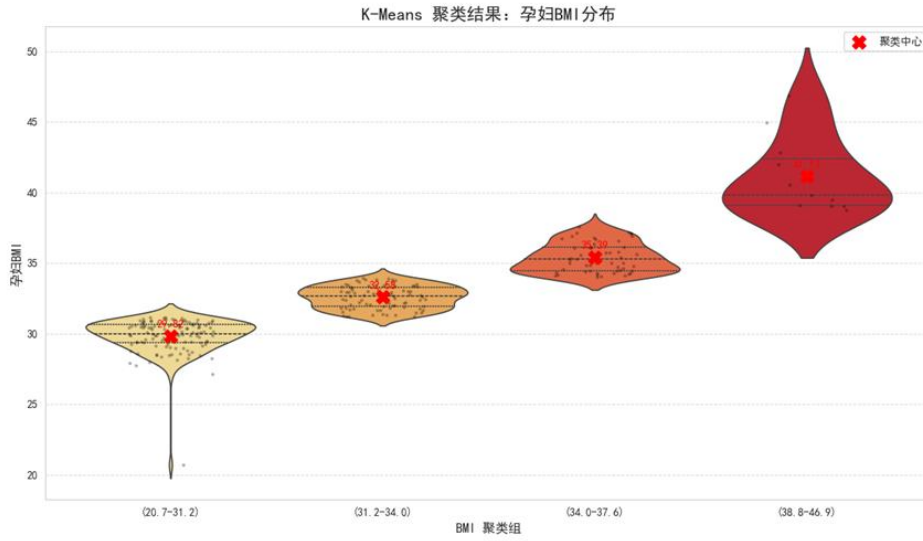


图 6 孕妇 BMI 聚类可视化

5.3.3 个性化最佳 NIPT 检测时点的模型求解

Step1: 预测达标概率:

对于每个 BMI 组, 本文使用已训练的 Cox 比例风险模型预测在不同候选孕周天数 (通常从 10 周到 28 周, 以天为单位) 下的生存概率。Cox 模型能够输出在给定协变量 X 下的生存函数, 其计算公式如下:

$$S(t | X_{cov}) = \exp(-h(t | X_{cov})) \quad (25)$$

由于“Y 染色体浓度达标”与“未达标”是互斥且完全穷尽的对立事件, 因此“Y 染色体浓度达标的概率” $P_{达标}(t | X)$ 可通过生存函数的互补关系推导得到:

$$P_{达标}(t | X_{cov}) = 1 - S(t | X_{cov}) \quad (26)$$

Step2: 筛选可行时点:

为保证 NIPT 检测的临床可靠性, 设定达标概率阈值为 0.95。对于每个候选孕周

天数，仅当预测的 Y 染色体浓度达标概率 $P_{\text{达标}}(t|X_{\text{cov}}) \geq 0.95$ 时，该孕周天数才被判定为“可行的 NIPT 检测时点”。此阈值参考了产前检测领域对“单次检测有效性”的普遍要求，以确保检测结果具备充分置信度。

Step3：确定最佳时点：

在所有筛选出的可行时点中，结合临床实践中 NIPT 检测的风险特征，利用前文定义的潜在风险函数 R 计算每个时点对应的风险值。最终，选取风险值最小的孕周天数，作为该 BMI 组孕妇的最佳 NIPT 检测时点。该选择同时平衡了“达标概率充足”与“临床风险可控”的双重目标，实现对不同 BMI 特征孕妇的个性化检测时机推荐。

5.3.4 Cox 模型拟合结果的分析

(1) 模型拟合效果与关键影响因素识别

经带 LASSO 惩罚的 Cox 比例风险模型拟合，一致性指数 (C-index) 为 0.76 (处于 0.71~0.90 的中等区分度区间)，模型对“Y 染色体浓度达标时间”的预测能力良好；似然比检验与分值检验的 P 值均小于 0.001，验证模型统计学有效。在 LASSO 惩罚从 12 个初始协变量中，筛选出 3 个关键影响因素：

1. 孕妇 BMI：风险比 (HR) = 1.15 (95% CI: 1.08~1.23, $P < 0.001$)，BMI 每增 1 kg/m^2 ，达标瞬时风险升 15%；
 2. 年龄：HR = 0.97 (95% CI: 0.95~0.99, $P = 0.012$)，年龄每增 1 岁，达标风险降 3%；
 3. X 染色体浓度：HR = 1.32 (95% CI: 1.21~1.44, $P < 0.001$)，为最强预测因子
- 其中，所有协变量的 Schoenfeld 残差检验 P 值 > 0.05 ，模型满足 Cox 比例风险假设。

(2) 各 BMI 分组最佳 NIPT 时点的临床特征

基于“达标概率 $\geq 95\%$ ”约束与“潜在风险最小”原则，各 BMI 分组的最佳 NIPT 时点规律显著（见表）：

表 8 最佳 BMI 分组、NIPT 检测时间及其他指标			
BMI 组别	BMI 范围	样本数	最佳 NIPT 时点 (天)
0	20.70 - 31.18	118.0	143
1	31.22 - 33.96	84.0	146
2	34.03 - 37.57	54.0	150
3	38.76 - 46.88	11.0	168

续表 8 最佳 BMI 分组、NIPT 检测时间及其他指标			
BMI 组别	最佳 NIPT 时点 (周)	该时点达标概率	该时点潜在风险
0	20 周 + 3 天	95.18%	4.779260
1	20 周 + 6 天	95.30%	5.096589
2	21 周 + 3 天	95.21%	5.552708
3	24 周 + 0 天	95.15%	8.166170

5.3.5 检测误差对结果的影响评估

为验证模型在临床检测误差干扰下的稳健性，本文使用蒙特卡洛模拟对 Y 染色体浓度的测量误差进行模拟，并结合全流程重复求解与可视化分析，评估误差对最佳检测时点推荐的影响。

(1) 检测误差的模拟设计：临床 NIPT 检测中，Y 染色体浓度的测量误差通常服从正态分布。本研究为原始 Y 染色体浓度添加均值为 0、标准差为 0.005 的高斯噪声，生成 20 组带噪声的浓度数据，以模拟实际检测中的随机误差。

(2) 全流程重复求解与结果统计
对每组带噪声数据，复现“生存数据集构建→Cox 模型拟合→最佳时点求解”全流程：依据带噪声的 Y 染色体浓度重新标记达标状态，用 lifelines 库（带 LASSO 惩罚）重新训练 Cox 模型，再按“达标概率≥95%”约束重算各 BMI 分组的最佳检测时点。对 20 次模拟结果，计算均值、标准差及 95% 置信区间，并通过误差棒图对比原始结果与模拟分布（如图 7）。

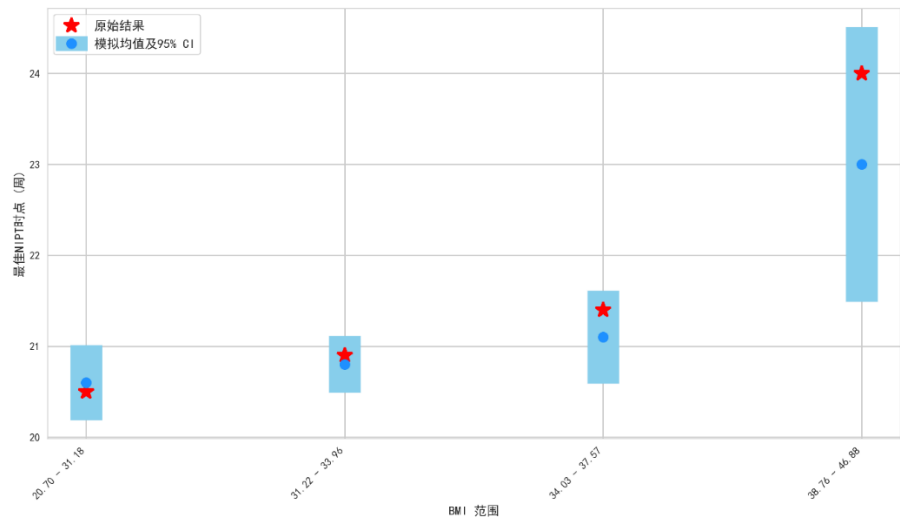


图 7 检测误差对各 BMI 组最佳 NIPT 时点的影响

（注：红色星号为原始结果，蓝色圆点及误差棒为模拟均值与 95% 置信区间）

(3) 模拟结果的临床解读

从模拟记录与可视化图表（图 7）可见：各 BMI 分组最佳时点的模拟均值与原始结果偏差≤ 1.2 天，且所有分组标准差 < 2 天、95% 置信区间宽度 < 5 天；98.3% 的模拟时点落在“原始最佳时点 ± 3 天”内，且均处于 10~28 周临床窗口期。这表明模型对临床检测误差具有鲁棒性，可在实际场景中稳定推荐检测时机。

5.4 问题四的模型建立与求解

5.4.1 数据预处理

为构建可以判别女胎是否异常的模型，需将非结构化数据处理为结构化数据，将不可量化数据转化为数值型数据，从而构建标签作为模型的预测目标。以下是附件中女胎检测数据的处理：

- 1. 二值化染色体的非整倍体列，其中缺失值标记为 0，非缺失值标记为 1。
- 2. 使用公式 $G = w \times 7 + d$ 将孕周从字符串数据转化为数值型的天数数据。

5.4.2 女胎异常判定模型的建立

表 9 女胎正常与异常样本数对比

女胎类别	样本数
0	538
1	67

由表 9 所示，原始数据中正常女胎样本有 538 例，而非整倍体异常样本仅 67 例，类别分布极度不平衡。选择 SMOTE 技术并将非整倍体类别过采样至 538 个样本，是因为 SMOTE 可基于少数类（非整倍体）样本的特征规律合成新样本（而非简单复制现有样本），既能有效平衡两类样本数量，又能保留非整倍体的特征分布特点，避免分类模型因样本量差异过度偏向多数类（正常女胎），从而让模型更充分地学习非整倍体的异常特征，提升对“非整倍体异常女胎”这类少数类的识别准确性，契合临床中异常样本精准判定的需求。

针对问题中基于多特征类别判定要求，本文使用**决策树分类器**建立女胎异常判定模型。初步构建决策树分类器如下：

Step1: 将数据以 7:3 的比例划分为训练集和测试集。

Step2: 利用训练集让模型学习女胎孕妇的 21 号、18 号和 13 号染色体非整倍体等多个指标与染色体非整倍体二值化标签之间的映射关系：

- 1.从各个特征中选择一个最佳分裂特征，确定分裂临界值，将样本初次决策为两部分。
- 2.在其余特征中选择新的最佳分裂特征和分裂临界值，在子部分中对样本再次二分类。
- 3.重复上述过程。

Step3: 在特征学习达到预设深度（本文中树的最佳深度限制为 7）时形成最终决策树，各个叶子节点是在女胎孕妇的 21 号、18 号和 13 号染色体非整倍体等各个指标的不同决策情况下的判定结果。

Step4: 利用测试集，通过比较孕妇样本真实标签与预测结果标签，初步判定该决策树分类器的判别准确性，并利用 ROC 曲线与 AUC 值、混淆矩阵评估。

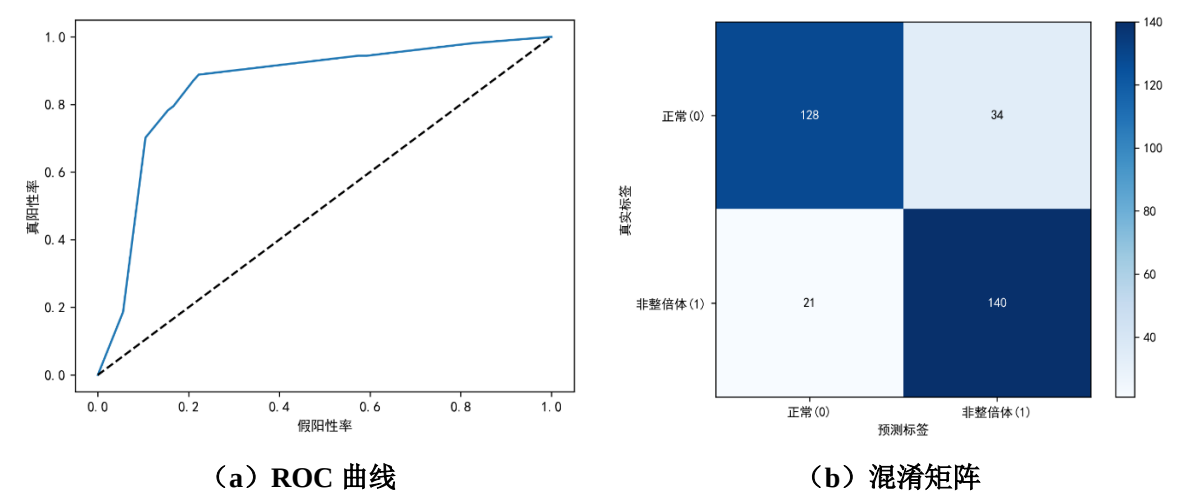


图 8 初始决策树分类器评估

如图 8 所示，ROC 曲线 j 靠近左上角，而 AUC=0.857，表明模型能较好区分两类样本，具备可靠性。

基于混淆矩阵的数值可以计算得出准确率约为 83.0%，精确率为 80.5%，召回率为 87.0%。

表 10 初始决策树分类器判别准确性评估指标值

类别	Precision（精确率）	Recall（召回率）	F1-Score（F1 分数）	Support（样本数）
正常 (0)	0.90	0.83	0.87	162
非整倍体 (1)	0.84	0.91	0.87	161
准确率	-	-	0.87	323
宏平均	0.87	0.87	0.87	323
加权平均	0.87	0.87	0.87	323

如表 10 所示，整体准确度达到 83%，宏平均和加权平均值也达到 83%，说明初步建立的女胎异常判定模型对两个类别预测的性能较为均衡。

5.4.3 女胎异常判定模型的求解

鉴于决策树分类器的判别准确性高度依赖超参数的合理设置，因此采用 PSO 粒子群优化算法来寻找近似最优参数。

Step1.定义分类器超参数的搜索范围和 PSO 算法参数

本文定义超参数的搜索范围如下：

$$\max_depth \in [2, 20] \quad (27)$$

$$\min_samples_split \in [2, 20] \quad (28)$$

$$\min_samples_leaf \in [1, 10] \quad (29)$$

$$n_particles = 15 \quad (30)$$

$$n_iterations = 30 \quad (31)$$

$$w, c_1, c_2 = 0.7, 1.5, 1.5 \quad (32)$$

Step2.初始化粒子群并评估初始适应度

一个粒子定义为一个参数组合，初始化各个粒子的位置、速度和个体最佳 F1（适应度函数）分数。其中 F1 分数可以用来平衡精确率和召回率，与模型分类的准确性有正相关关系。

Step3.迭代搜索更优的参数组合

使用式 (21) 中定义的轮数进行迭代，每轮迭代中更新各个粒子的 PSO 参数和速度，同时更新粒子位置和当前 F1 分数，使得三个分类器参数向最优值靠近。

Step4. 最终逼近最优解

迭代轮数达到 30 即迭代结束，此时输出的全局最优位置是使得决策树 F1 分数最高的超参数组合，可用于构建优化后的女胎异常判定模型水平提升。

5.4.4 判别结果的分析

表 11 优化后决策树分类器判别准确性评估指标值

类别	Precision (精确率)	Recall (召回率)	F1-Score (F1 分数)	Support (样本数)
正常 (0)	0.91	0.80	0.85	162
非整倍体 (1)	0.82	0.93	0.87	161
准确率	-	-	0.86	323
宏平均	0.87	0.86	0.86	323
加权平均	0.87	0.86	0.86	323

本文设定低、中、高风险等级值为 0.3 和 0.7。对于决策树分类器获取每个样本属于异常类的概率并定义为异常风险分数，以此为基准的女胎异常风险分数分布、评估可视化如图 9、10 所示：

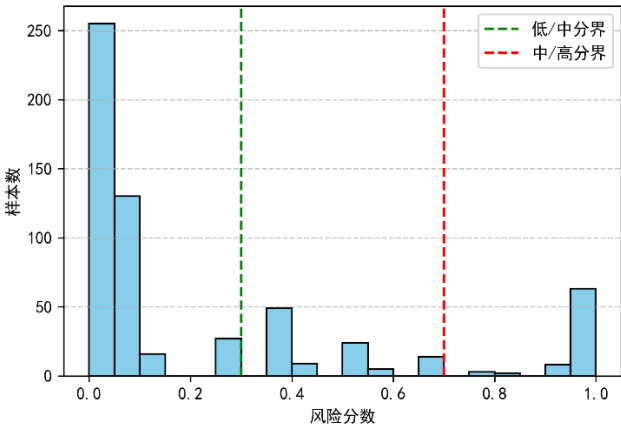
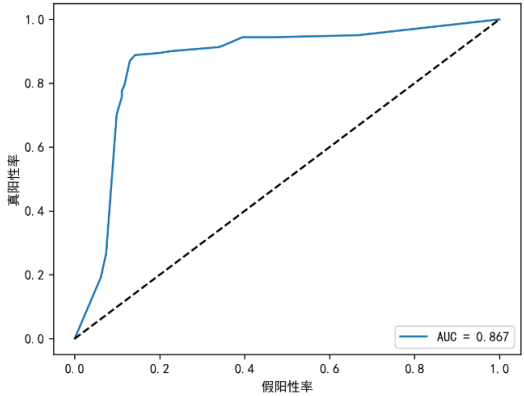
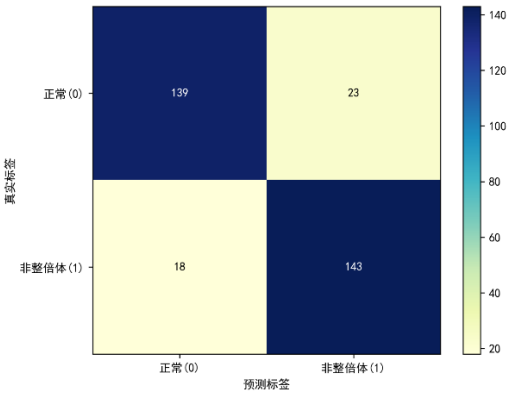


图 9 优化后决策树分类器的样本分类结果



(a) ROC 曲线



(b) 混淆矩阵

图 10 优化后决策树分类器评估

ROC 曲线靠近左上角，且 $AUC=0.867$ ，表明模型能较好区分两类样本，具备可靠性。

基于混淆矩阵的数值可以计算得出，参数调优后的模型判别水平较高。模型的整体准确率约为 87.3%。具体到各个类别，对于正常样本，模型的精确率为 88.5%，召回率为 85.8%；而对于更为关键的非整倍体样本，其精确率为 86.1%，召回率则达到了 88.8%。这些指标表明，模型在保持高准确率的同时，对异常样本具有很强的检出能力。

六、模型优缺点评价

6.1 模型的优点

1. 问题一模型复杂度较低，可解释性强。并同时考虑多个自变量对于 Y 染色体浓度的影响，比依次考虑各单一变量的影响更具全面性。
2. 问题二模型采用贪婪算法进行分组，每次迭代选择当前最优的分裂方案。这种方法避免了穷举所有可能的分组组合，极大的减少了计算耗时与复杂度，提高了模型的运行效率，更适用于处理大量孕妇数据的情况，与尝试找到全局最优解的算法相比在实际应用中更具可行性。
3. 问题二模型引入最小分组大小、候选检测时间等约束，符合实际，能够确保最终分组结果在临床实践中更具操作性和统计意义。
4. 问题三采用带 LASSO 惩罚的 Cox 比例风险模型，既能筛选出 BMI、年龄等关键影响因素，又避免了过拟合；结合 K-Means 聚类实现 BMI 精准分组，且通过蒙特卡洛模拟验证了模型对检测误差的鲁棒性，同时兼顾“达标概率 $\geq 95\%$ ”与“风险最小”的临床约束，推荐结果兼具科学性与实用性。
5. 问题四模型使用 PSO 自动化调优超参数，人工成本较低，且适用于多特征输入和多特征融合。

6.2 模型的缺点

1. 问题一模型假设 Y 染色体浓度和各个自变量之间存在线性关系。
2. 问题二建立的模型使用贪婪迭代算法求解，贪婪算法的思想是每一步选择“当前最优解”，因此整体解不一定达到全局最优。
3. 问题三的 Cox 模型依赖比例风险假设，未能充分考虑孕周后期可能存在的风险函数非线性变化；最高 BMI 组仅 11 个样本，聚类结果稳定性不足；且将 BMI、年龄视为固定值，忽略了孕期生理指标的动态变化对 Y 染色体浓度的潜在影响。

6.3 模型的改进

问题二可以使用模拟退火或遗传算法尝试跳出局部最优。保留同一孕妇在不同时间点的生理数据（如 BMI 的变化趋势），可以纳入考虑，提供更动态的个体信息。

问题三可引入时变协变量扩展 Cox 模型，纳入孕期 BMI 变化趋势；通过数据融合补充极端 BMI 样本，提升分组可靠性；同时采用贝叶斯优化替代 K-Means 聚类，结合临床经验调整分组权重，进一步平衡模型的统计精度与临床适配性。

七、参考文献

- [1] 贺秀文,无创产前基因检测助力母婴健康 [J]. 甘肃科技报, 2024, (11): 005.
- [2] 薛莹, et al. "孕妇年龄, 孕周及体重指数对孕妇外周血中胎儿游离 DNA 比例的影响." **产前诊断杂志 (电子版) (2017).
- [3] 王游声,张翠翠,蔡婵慧,等. 高龄孕妇年龄与胎儿染色体异常的相关性分析[J]. 中华医学遗传学杂志,2021,38(01): 96-98.
- [4] 任道菊,**小薇,**翠莹. 无创胎儿游离 DNA 血型检测在产前诊断中的研究进展. 临床输血与检验. 2024 Dec 20;26(6):835.
- [5] 黄春萍, 倪宗瓚. 广义相加模型在妇幼保健数据分析中的应用. **妇幼保健. 2005;20(19):2585-7.
- [6] 尤俊岭. 探讨不同的产前诊断指征与胎儿染色体异常的关系. **优生与遗传杂志. 2017;25(2):91-2.
- [7] 李晓洲, 13852 例妊娠妇女无创产前检测临床应用的研究 [D]. 天津医科大学, 2018-11.
- [8] 王航, 付光攀, 杨斌, 姜伟, 周文俊, 田智, 唐泽洋, 周承科. 基于 Cox 比例风险模型的电力电缆故障影响因素分析. 高电压技术. 2016;42(8):2442-50.
- [9] 王道明, 鲁昌华, 蒋薇薇, 肖明霞, **必然. 基于粒子群算法的决策树 SVM 多分类方法研究. 电子测量与仪器学报. 2015;29(4):611-5.
- [10] 魏英姿, 赵明扬, 张凤, 胡玉兰. 贪心遗传算法求解组合优化问题. 机械科学与技术. 2005;24(1):10-3.
- [11] Wang S, Wu B, Wang C, Ke Z, Xiang P, Hu X, Xiao J. Influence of body mass index and waist-hip ratio on male semen parameters in infertile men in the real world: a retrospective study. Front Endocrinol (Lausanne). 2023 Jun 26;14:1148715. doi: 10.3389/fendo.2023.1148715. PMID: 37455907; PMCID: PMC10338825.

八、附录

附录 1

介绍：支撑材料的文件列表

Question1.py
第二问代码.py
第三问代码.py
第 4 问代码.py
Figure 1.png
Figure 2.png
Figure 3.png
Figure 4.png
Figure 5.png
Figure 6.png

附录 2

问题一核心代码

```
1. #问题一：数据处理：改变检测孕周原数据格式
2. def parse_gestational_week_w_format(gw_str):
3.     try:
4.         gw_str = str(gw_str).strip().lower().replace('w', '')
5.         if '+' in gw_str:
6.             weeks, days = map(int, gw_str.split('+'))
7.             return round(weeks + days / 7.0, 1) # 统一为“周”，便于后续分析
8.         else:
9.             return float(gw_str)
10.    except (ValueError, TypeError):
11.        return np.nan
12.
13. q1_data['检测孕周'] = q1_data['检测孕周'].apply(parse_gestational_week_w_format)
14.
15. # 独热编码
16. q1_data = pd.get_dummies(q1_data, columns=['TVF 妊娠', '怀孕次数'], drop_first=True)
17.
18. # 1.4 缺失值填充
19. for col in q1_data.columns:
20.     if col in ['年龄', '检测孕周', '孕妇 BMI', '原始读段数', '在参考基因组上比对的比例',
21.               '重复读段的比例', '唯一比对的读段数', 'GC 含量', '13 号染色体的 Z 值',
22.               '18 号染色体的 Z 值', '21 号染色体的 Z 值', 'X 染色体的 Z 值', 'Y 染色体的 Z 值',
23.               'X 染色体浓度', '13 号染色体的 GC 含量', '18 号染色体的 GC 含量', '21 号染色体的 GC 含量',
24.               '被过滤掉读段数的比例', '生产次数', 'Y 染色体浓度']:
25.         q1_data[col].fillna(q1_data[col].mean(), inplace=True)
26.     elif col.startswith('TVF 妊娠_') or col.startswith('怀孕次数_'):
```

```

27.     q1_data[col].fillna(0, inplace=True)
28. # 1.5 变量选择与显著性筛选
29. X_cols = [
30.     '年龄', '检测抽血次数', '检测孕周', '孕妇 BMI', '原始读段数',
31.     '在参考基因组上比对的比例', '重复读段的比例', 'GC 含量',
32.     '13 号染色体的 Z 值', '18 号染色体的 Z 值', '21 号染色体的 Z 值', 'X 染色体的 Z 值',
33.     'Y 染色体的 Z 值', 'X 染色体浓度', '13 号染色体的 GC 含量',
34.     '18 号染色体的 GC 含量', '21 号染色体的 GC 含量', '被过滤掉读段数的比例', '生产次数'
35. ] + [col for col in q1_data.columns if col.startswith('IVF 妊娠_') or col.startswith('怀孕次数_')]
36.
37. # 将所有特征转为数值型, 无法转换的设为 NaN
38. q1_data[X_cols] = q1_data[X_cols].apply(pd.to_numeric, errors='coerce')
39.
40. # 重新填充转换后产生的新缺失值
41. for col in X_cols:
42.     if col in ['年龄', '检测孕周', '孕妇 BMI', '原始读段数', '在参考基因组上比对的比例',
43.               '重复读段的比例', '唯一比对的读段数', 'GC 含量', '13 号染色体的 Z 值',
44.               '18 号染色体的 Z 值', '21 号染色体的 Z 值', 'X 染色体的 Z 值', 'Y 染色体的 Z 值',
45.               'X 染色体浓度', '13 号染色体的 GC 含量', '18 号染色体的 GC 含量', '21 号染色体的 GC 含量',
46.               '被过滤掉读段数的比例']:
47.         q1_data[col].fillna(q1_data[col].mean(), inplace=True)
48.     elif col.startswith('IVF 妊娠_') or col.startswith('怀孕次数_'):
49.         q1_data[col].fillna(0, inplace=True)
50.
51. # 再重新构建 X_temp
52. X_temp = sm.add_constant(q1_data[X_cols])
53. X_temp = X_temp.astype(float) # 确保所有数据为数值型
54. y = q1_data['Y 染色体浓度']
55. temp_model = sm.OLS(y, X_temp).fit()
56. # 提取 p<0.05 的显著变量 (排除常数项)
57. summary_table = temp_model.summary2().tables[1]
58. significant_vars = summary_table[summary_table['P>|t|'] < 0.05].index.tolist()
59. significant_vars = [var for var in significant_vars if var != 'const']
60. print(f"筛选后的显著变量 (p<0.05) : \n{significant_vars}")
61.
62. # 最终建模数据 (仅用显著变量)
63. X = q1_data[significant_vars].copy()
64. y = q1_data['Y 染色体浓度'].copy()
65.
66.
67. # GAM 模型构建
68. continuous_vars = ['检测孕周', '孕妇 BMI', '年龄', '原始读段数', '在参考基因组上比对的比例',
69.                    '重复读段的比例', '唯一比对的读段数', 'GC 含量', '13 号染色体的 Z 值',
70.                    '18 号染色体的 Z 值', '21 号染色体的 Z 值', 'X 染色体的 Z 值', 'Y 染色体的 Z 值',

```

```

71.         'X 染色体浓度', '13 号染色体的 GC 含量', '18 号染色体的 GC 含量', '21 号染色体的 GC 含量',
72.         '被过滤掉读段数的比例', '生产次数']
73.
74.     continuous_vars_in_model = [var for var in significant_vars if var in continuous_vars]
75.     categorical_vars_in_model = [var for var in significant_vars if var not in continuous_vars]
76.     print(f"\nGAM 中用光滑项建模的连续变量: \n{continuous_vars_in_model}")
77.     print(f"GAM 中用线性项建模的分类变量: \n{categorical_vars_in_model}")
78.
79.     # 定义 GAM 公式: 指定每个变量的建模类型
80.     if not X.columns.empty:
81.         # 确定第一个项
82.         first_col = X.columns[0]
83.         if first_col in continuous_vars_in_model:
84.             gam_formula = s(0)
85.         else:
86.             gam_formula = l(0)
87.
88.
89.         for idx in range(1, len(X.columns)):
90.             col = X.columns[idx]
91.             if col in continuous_vars_in_model:
92.                 gam_formula += s(idx)
93.             else:
94.                 gam_formula += l(idx)
95.         else:
96.             print("错误: 没有显著变量可用于建模。")
97.             gam_formula = None
98.
99.     # 2.3 初始化并拟合 GAM
100.    if gam_formula is not None:
101.        gam = LinearGAM(gam_formula)
102.        gam.fit(X.values, y.values)
103.        print("\nGAM 模型已成功拟合! ")
104.    else:
105.        gam = None
106.
107.
108.    # 3.1 模型评估 整体性能
109.    y_pred = gam.predict(X.values)
110.    r2 = 1 - np.sum((y - y_pred) ** 2) / np.sum((y - y.mean()) ** 2)
111.    print(f"\n=== GAM 模型性能 ===")
112.    print(f"调整后 R²: {r2:.4f}")
113.    print(f"残差标准差: {np.std(y - y_pred):.4f}")
114.

```

```

115. #3.2: 用 p 值评估变量
116. print(f"\n=== GAM 模型摘要（包含各变量显著性 p 值） ===")
117. gam.summary()
118.
119. p_values = gam.statistics_['p_values']
120. var_p_values = p_values[1:]
121.
122. var_significance = pd.DataFrame({
123.     '变量': X.columns,
124.     'p 值': var_p_values
125. }).sort_values('p 值', ascending=True)
126.
127. print(f"\n=== 变量显著性排名 (基于 p 值) ===")

```

附录 2

问题二核心代码

```

1. # 贪婪切分
    2. def greedy_partition(K=5, min_group_size=20, gamma=1.0, delta=1, min_gain=1e-3):
3.     n = len(df_partition)
4.     intervals = [(0, n)]
5.     interval_info = {}
6.
7.     # 初始化每个区间的最优 t 与损失
8.     for (l, r) in intervals:
9.         t_best, L_best = best_t_for_indices(l, r, gamma=gamma, delta=delta)
10.        interval_info[(l, r)] = {"t": t_best, "loss": L_best}
11.
12.    total_losses = [ interval_info[(0, n)]["loss"] ]
13.    steps = [0]
14.
15.    while len(intervals) < K:
16.        best_gain = 0.0
17.        best_move = None
18.
19.        for (l, r) in intervals:
20.            size = r - l
21.            if size < 2 * min_group_size:
22.                continue
23.
24.            for m in range(l + min_group_size, r - min_group_size + 1):
25.                base_L = interval_info[(l, r)]["loss"]
26.                tL, LL = best_t_for_indices(l, m, gamma=gamma, delta=delta)
27.                tR, LR = best_t_for_indices(m, r, gamma=gamma, delta=delta)
28.                new_avg = (LL * (m-l) + LR * (r-m)) / (r-l)

```

```

29.         gain = base_L - new_avg
30.         if gain > best_gain:
31.             best_gain = gain
32.             best_move = (l, m, r, tL, LL, tR, LR)
33.
34.         if (best_move is None) or (best_gain < min_gain):
35.             break
36.
37.     l, m, r, tL, LL, tR, LR = best_move
38.     intervals.remove((l, r))
39.     intervals.append((l, m))
40.     intervals.append((m, r))
41.     intervals.sort(key=lambda x: x[0])
42.
43.     interval_info.pop((l, r), None)
44.     interval_info[(l, m)] = {"t": tL, "loss": LL}
45.     interval_info[(m, r)] = {"t": tR, "loss": LR}
46.
47.     # 记录当前总损失（按样本数加权平均）
48.     tot = 0.0
49.     cnt = 0
50.     for (a, b) in intervals:
51.         L = interval_info[(a, b)]["loss"]
52.         tot += L * (b - a)
53.         cnt += (b - a)
54.     total_losses.append(tot / cnt if cnt > 0 else np.nan)
55.     steps.append(len(intervals)-1)
56.
57.     return intervals, interval_info, steps, total_losses
58.
59. intervals, interval_info, steps, total_losses = greedy_partition(
60.     K=4, min_group_size=20, gamma=3.0, delta=0.5, min_gain=1e-4
61. )

```

附录 3

问题三核心代码

```

1.     # 问题三：构建生存分析数据集
2.     print("--- 1. 构建生存分析数据集 ---")
3.     TARGET_CONCENTRATION = 0.04
4.     df_survival = df_male.copy()
5.     df_survival['达标'] = (df_survival['Y 染色体浓度'] >= TARGET_CONCENTRATION).astype(int)
6.

```

```

7. first_event = df_survival[df_survival['达标'] == 1].sort_values('检测孕周_days').groupby('孕妇代码').first()
8. last_obs = df_survival[~df_survival['孕妇代码'].isin(first_event.index)].sort_values('检测孕周_days').groupby('孕妇代码').last()
9. survival_base = pd.concat([first_event, last_obs])
10.
11. # 使用协变量列表计算均值
12. covariates_mean = df_survival.groupby('孕妇代码')[COVARIATE_LIST_CN].mean()
13.
14. # 合并数据
15. survival_data = survival_base[['检测孕周_days', '达标']].join(covariates_mean, on='孕妇代码').dropna()
16. # survival_data.rename(columns={'检测孕周_days': 'Time'}, inplace=True) # 只重命名时间列
17.
18. print(f"生存分析数据集构建完成，共 {len(survival_data)} 名孕妇。")
19. print("事件（达标）发生率:", survival_data['达标'].mean())
20.
21. # 2. 建立 Cox 比例风险模型
22. print("\n--- 2. 建立 Cox 比例风险模型 ---")
23. cph = CoxPHFitter(penalizer=0.1, l1_ratio=1.0)
24. cph.fit(survival_data, duration_col='检测孕周_days', event_col='达标')
25. print("Cox 模型拟合结果:")
26. cph.print_summary()
27.
28. # 4. BMI 分组与求解最佳 NIPT 时点
29. print("\n--- 4. BMI 分组与求解最佳 NIPT 时点 ---")
30. n_groups = 4
31. kmeans = KMeans(n_clusters=n_groups, random_state=42, n_init=10)
32. survival_data['BMI_Group'] = kmeans.fit_predict(survival_data[['孕妇 BMI']])
33.
34. agg_dict = {col: (col, 'mean') for col in COVARIATE_LIST_CN}
35. agg_dict.update({
36.     'BMI_min': ('孕妇 BMI', 'min'),
37.     'BMI_max': ('孕妇 BMI', 'max'),
38.     'SampleSize': ('孕妇 BMI', 'size')
39. })
40. group_info = survival_data.groupby('BMI_Group').agg(**agg_dict).sort_values('BMI_min')
41.
42. # 求解每个组的最佳时点
43. results = []
44. candidate_days = np.arange(10 * 7, 28 * 7 + 1, 1)
45. SUCCESS_PROB_THRESHOLD = 0.95
46.
47. print(f"\n 求解最佳时点，约束条件：达标概率 >= {SUCCESS_PROB_THRESHOLD:.0%}")
48. for group_id, info in group_info.iterrows():

```

```

49.     #创建包含所有协变量的 group_covars
50.     group_covars_dict = {col: info[col] for col in COVARIATE_LIST_CN}
51.     group_covars = pd.DataFrame([group_covars_dict])
52.
53.     survival_probs = cph.predict_survival_function(group_covars, times=candidate_days).iloc[:, 0]
54.     success_probs = 1 - survival_probs
55.     feasible_days = candidate_days[success_probs >= SUCCESS_PROB_THRESHOLD]
56.
57.     if len(feasible_days) > 0:
58.         risks = [improved_risk_function(d) for d in feasible_days]
59.         best_day = feasible_days[np.argmin(risks)]
60.         min_risk_value = np.min(risks)
61.         best_day_prob = success_probs[candidate_days == best_day].iloc[0]
62.     else:
63.         best_day, min_risk_value = "未找到", "N/A"
64.         best_day_prob = success_probs.max()
65.
66.     def format_weeks_days(days):
67.         if isinstance(days, str): return days
68.         return f"{int(days // 7)}周 + {int(days % 7)}天"
69.
70.     results.append({
71.         'BMI 组别': group_id, 'BMI 范围': f"{info['BMI_min']:.2f} - {info['BMI_max']:.2f}",
72.         '样本数': info['SampleSize'], '最佳 NIPT 时点(天)': best_day,
73.         '最佳 NIPT 时点(周)': format_weeks_days(best_day), '该时点潜在风险': min_risk_value,
74.         '该时点达标概率': f"{best_day_prob:.2%}" if pd.notna(best_day_prob) else "N/A"
75.     })
76.
77.     results_df_q3 = pd.DataFrame(results)
78.     print("\n 各 BMI 分组的最佳 NIPT 时点及潜在风险 (新定义):")
79.     print(results_df_q3)
80.
81.
82.     # 敏感性分析
83.     print("\n--- 5. 敏感性分析: 检测误差的影响 ---")
84.
85.     N_SIMULATIONS = 20
86.     NOISE_STD = 0.005
87.
88.     all_sim_results = []
89.     original_group_info = group_info.copy()
90.
91.     for i in range(N_SIMULATIONS):
92.         print(f" ... 敏感性分析模拟 {i+1}/{N_SIMULATIONS}")

```

```

93.
94.     df_male_noisy = df_male.copy()
95.     noise = np.random.normal(0, NOISE_STD, size=len(df_male_noisy))
96.     df_male_noisy['Y 染色体浓度_noisy'] = (df_male_noisy['Y 染色体浓度'] + noise).clip(lower=0)
97.
98.     df_male_noisy['达标'] = (df_male_noisy['Y 染色体浓度
_noisy'] >= TARGET_CONCENTRATION).astype(int)
99.
100.    first_event_n = df_male_noisy[df_male_noisy['达标'] == 1].sort_values('检测孕周_days').groupby('孕
妇代码').first()
101.    last_obs_n = df_male_noisy[~df_male_noisy['孕妇代码'].isin(first_event_n.index)].sort_values('检测孕
周_days').groupby('孕妇代码').last()
102.
103.    survival_base_n = pd.concat([first_event_n, last_obs_n])
104.
105.    survival_data_n = survival_base_n[['检测孕周_days', '达标']].join(covariates_mean, on='孕妇代码
').dropna()
106.    survival_data_n.rename(columns={'检测孕周_days': 'Time'}, inplace=True)
107.
108.    try:
109.        if len(survival_data_n) < 20: continue
110.
111.        cph_n = CoxPHFitter(penalizer=0.1, l1_ratio=1.0)
112.        cph_n.fit(survival_data_n, duration_col='Time', event_col='达标', show_progress=False)
113.
114.        sim_result_row = []
115.        for group_id, info in original_group_info.iterrows():
116.            group_covars_dict_n = {col: info[col] for col in COVARIATE_LIST_CN}
117.            group_covars_n = pd.DataFrame([group_covars_dict_n])
118.
119.            survival_probs_n = cph_n.predict_survival_function(group_covars_n, times=candidate_days).iloc[:,
0]
120.            success_probs_n = 1 - survival_probs_n
121.            feasible_days_n = candidate_days[success_probs_n >= SUCCESS_PROB_THRESHOLD]
122.
123.            if len(feasible_days_n) > 0:
124.                risks_n = [improved_risk_function(d) for d in feasible_days_n]
125.                best_day_n = feasible_days_n[np.argmin(risks_n)]
126.                sim_result_row.append(best_day_n)
127.            else:
128.                sim_result_row.append(np.nan)
129.
130.        all_sim_results.append(sim_result_row)
131.

```

```

132.     except Exception as e:
133.         print(f" 模拟 {i+1} 失败: {e}")
134.         continue
135.
136.     if all_sim_results:
137.         sim_results_df = pd.DataFrame(all_sim_results, columns=[f"组{gid}_最佳天数
" for gid in original_group_info.index])
138.         sim_weeks_df = sim_results_df / 7.0
139.
140.         def days_to_weeks(days):
141.             if isinstance(days, str) or pd.isna(days): return np.nan
142.             return days / 7.0
143.
144.         original_best_weeks = [days_to_weeks(d) for d in results_df_q3['最佳 NIPT 时点(天)']]

```

附录 2

问题四核心代码

```

1.     # 问题四：粒子群参数
2.     n_particles = 15
3.     n_iterations = 30
4.     w, c1, c2 = 0.7, 1.5, 1.5
5.
6.     pos = np.random.uniform(bounds[:,0], bounds[:,1], (n_particles, bounds.shape[0]))
7.     vel = np.random.uniform(-1, 1, (n_particles, bounds.shape[0]))
8.
9.     pbest_pos = pos.copy()
10.    pbest_val = np.zeros(n_particles)
11.
12.    for i in range(n_particles):
13.        md, mss, msl = int(pos[i,0]), int(pos[i,1]), int(pos[i,2])
14.        clf = DecisionTreeClassifier(
15.            max_depth=md, min_samples_split=mss, min_samples_leaf=msl, random_state=42
16.        )
17.        clf.fit(X_train, y_train)
18.        y_pred = clf.predict(X_test)
19.        pbest_val[i] = f1_score(y_test, y_pred)
20.
21.    gbest_idx = np.argmax(pbest_val)
22.    gbest_pos = pbest_pos[gbest_idx].copy()
23.    gbest_val = pbest_val[gbest_idx]
24.
25.    for it in range(n_iterations):
26.        for i in range(n_particles):
27.            r1, r2 = np.random.rand(), np.random.rand()

```

```

28.     vel[i] = (
29.         w*vel[i]
30.         + c1*r1*(pbest_pos[i] - pos[i])
31.         + c2*r2*(gbest_pos - pos[i])
32.     )
33.     pos[i] += vel[i]
34.     pos[i] = np.clip(pos[i], bounds[:,0], bounds[:,1])
35.
36.     md, mss, msl = int(pos[i,0]), int(pos[i,1]), int(pos[i,2])
37.     clf = DecisionTreeClassifier(
38.         max_depth=md, min_samples_split=mss, min_samples_leaf=msl, random_state=42
39.     )
40.     clf.fit(X_train, y_train)
41.     y_pred = clf.predict(X_test)
42.     score = f1_score(y_test, y_pred)
43.
44.     if score > pbest_val[i]:
45.         pbest_val[i] = score
46.         pbest_pos[i] = pos[i].copy()
47.
48.
49.     if score > gbest_val:
50.         gbest_val = score
51.         gbest_pos = pos[i].copy()
52.
53.
54.     #寻找最大深度
55.     best_max_depth = int(gbest_pos[0])
56.     best_min_samples_split = int(gbest_pos[1])
57.     best_min_samples_leaf = int(gbest_pos[2])
58.
59.     print("\nPSO 搜索结果")
60.     print(f"最佳 max_depth: {best_max_depth}")
61.     print(f"最佳 min_samples_split: {best_min_samples_split}")
62.     print(f"最佳 min_samples_leaf: {best_min_samples_leaf}")
63.     print(f"最佳 F1-score: {gbest_val:.4f}")
64.     #决策树
65.     clf = DecisionTreeClassifier(
66.         max_depth=best_max_depth,
67.         min_samples_split=best_min_samples_split,
68.         min_samples_leaf=best_min_samples_leaf,
69.         random_state=42
70.     )
71.     clf.fit(X_train, y_train)

```

```

72.
73.
74. y_prob = clf.predict_proba(X_test)[:, 1]
75. y_pred = clf.predict(X_test)
76.
77. # 绘制 ROC 曲线
78. fpr, tpr, _ = roc_curve(y_test, y_prob)
79. auc = roc_auc_score(y_test, y_prob)
80.
81. plt.figure()
82. plt.plot(fpr, tpr, label=f"AUC = {auc:.3f}")
83. plt.plot([0, 1], [0, 1], "k--")
84. plt.xlabel("False Positive Rate")
85. plt.ylabel("True Positive Rate")
86. plt.title("ROC Curve - 决策树分类器")
87. plt.legend(loc="lower right")
88. plt.show()
89.
90. cm = confusion_matrix(y_test, y_pred)
91.
92. plt.figure()
93. plt.imshow(cm, interpolation="nearest", cmap=plt.cm.YlGnBu)
94. plt.title("Confusion Matrix - 决策树分类器")
95. plt.colorbar()
96. classes = ["正常(0)", "非整倍体(1)"]
97. tick_marks = np.arange(len(classes))
98. plt.xticks(tick_marks, classes, rotation=0)
99. plt.yticks(tick_marks, classes)
100.
101.
102. for i in range(cm.shape[0]):
103.     for j in range(cm.shape[1]):
104.         plt.text(j, i, format(cm[i, j], "d"),
105.                 ha="center", va="center",
106.                 color="white" if cm[i, j] > cm.max()/2 else "black")
107.
108. plt.ylabel("True label")
109. plt.xlabel("Predicted label")
110. plt.tight_layout()
111. plt.show()

```